

JavaScript

Chapitre 01

Les bases de JavaScript

Plan

- Préambule
- Historique
- Structure lexicale
- Variables
- Types de données
- Instructions
- Concepts de programmation orientée objets et JavaScript

Préambule

Les bases de JavaScript

Trois idées fausses

JavaScript n'est pas Java

JavaScript n'est pas simple

JavaScript n'est pas un langage restreint aux navigateurs web !

Historique

Langage de programmation de scripts créé en 1995 par Brendan Eich pour la société Netscape

Initialement Live Script

Le 4 décembre 1995 devient JavaScript

Code intégralement interprété par le navigateur (coté client)

Visible par les messages d'erreurs qui s'affiche dans la console du navigateur.

Devient rapidement un standard de fait

Passage d'un standard de fait à une définition normalisée : ECMAScript

JavaScript = Implantation JavaScript de ECMAScript (JScript pour IE, SpiderMonkey pour Mozilla, etc..)

ActionScript est aussi une implantation de ECMAScript (l'une des plus fidèle en fait)

Les différentes éditions d'ECMAScript

ECMAScript de 1997 à nos jours

Year	Name	Description
1997	ECMAScript 1	First Edition
1998	ECMAScript 2	Editorial changes only
1999	ECMAScript 3	Added Regular Expressions. Added try/catch
1999	ECMAScript 4	Was never released
2009	ECMAScript 5	Added "strict mode". Added JSON support
2011	ECMAScript 5.1	Editorial changes
2015	ECMAScript 6	Added classes and modules
2016	ECMAScript 2016	Added exponential operator (**). Added Array.prototype.includes

Year	Name	Description
2017	ECMAScript 2017	Added Object.entries, Object.values, async functions

Javascript, langage interprété

Interprétation et compilation

JavaScript est un langage interprété *i.e.* le code est exécuté de manière séquentielle et le résultat du code exécuté est envoyé immédiatement. Le code n'a pas besoin de passer par une étape de transformation en exécutable pour être exécuté comme cela serait le cas pour un langage compilé.

Une question d'interpréteur

Historiquement, chaque navigateur possédait son propre interpréteur (SpiderMonkey chez Mozilla, Chakra chez Microsoft, Carakan pour Opera, etc.). Actuellement V8 prédomine le marché et intègre une grande majorité de systèmes.

Optimisation

En réalité, afin d'optimiser son exécution, la majorité des interpréteurs passent par une phase de précompilation durant l'exécution. Ainsi, si un même morceau de codes doit être réexécuté, cela sera plus rapide.

Single threaded

Javascript est un langage Single-threaded. Cela signifie que les actions sont exécutées les unes après les autres, jamais en parallèle.

Le moteur Javascript repose sur deux éléments :

- La **Memory Heap** : C'est la partie qui gère l'allocation dynamique de la mémoire ;
- La **Call Stack** : C'est une pile d'exécution. Elle contient les données des fonctions exécutées par le programme.

Remarque :

le fait d'avoir qu'une seule pile d'exécution simplifie le traitement mais en contre partie cela implique que le programme ne s'exécute que sur un seul thread. Ainsi, si une fonction est particulièrement longue à s'afficher, cela bloque l'intégralité du programme.

aller plus loin :

<https://www.axopen.com/blog/2020/08/javascript-langage-compile/>

Structure lexicale

Les bases de JavaScript

Structure lexicale

Jeux de caractères Unicode (codage sur 16 bits)

ECMAScript v3 : partout dans le code

ECMAScript v1 et v2 : uniquement dans les commentaires, ASCII dans le code

Casse

Langage sensible à la casse : **toto** ≠ **Toto** ≠ **TOTO**

Points-virgules

Chaque instruction se termine par un point-virgule **;**.

```
var i=3;
```

Celui-ci est facultatif lorsque les instructions sont séparées par des sauts de ligne.

```
var a=3  
var b=4
```

Mais attention à cette pratique !

aller plus loin : **<https://standardjs.com/>**

Commentaires

Les commentaires sont précédés de `//` pour une seule ligne,
ou entre `/*` et `*/` pour un commentaire sur plusieurs lignes.

```
// Ceci est une ligne de commentaire
/* Ceci est aussi un commentaire */
/*
 * Un commentaire sur
 * plusieurs lignes
 */
function carre(x) {
    return x*x; //retourne x2
}
```

Identifiants

Nom pour nommer une variable, fonction, étiqueter certaines boucles, etc.

Le 1er caractère doit être : une lettre, ou un souligné `_`, ou un dollar `$`.

Depuis ECMAScript v3, il est possible d'utiliser des lettres et des chiffres du jeu de caractères Unicode complet mais aussi des séquences d'échappement.

```
var π = 3.14;  
var \u03c0 = 3.14;
```

Mots réservés

Certains mots ne peuvent pas être utilisés comme identifiant, ce sont les mots réservés :

```
break
continue
delete
finally
if
new
this
typeof
while
case
debugger
do
for
in
return
throw
var
with
catch
default
else
function
```

```
instanceof  
switch  
try  
void
```

D'autres mots, non utilisés dans JavaScript, sont réservés dans la norme ECMAScript v3 :

```
class  
extends  
implements  
package  
public  
enum  
import  
interface  
private  
static  
export  
super  
protected  
yield
```

Expressions littérales

Un littéral est une expression apparaissant directement dans le programme. C'est un élément très important du langage.

Exemples de littéraux :

```
12 // Le nombre 12
1.2 // Le nombre 1.2
"salut" // Une chaîne de
texte
true // Une valeur
booléenne
/ab+c/i // Forme
littérale d'une expression régulière
null // Absence
d'objet
{l:5,h:3} //
Initialisation d'objet
[1,2,3,4,5] //
Initialisation de tableau
var carre= function(x) { return x*x;} // Fonction
littérale
```


Les variables

Les bases de JavaScript

Déclaration

Pour déclarer une variable, on utilise l'instruction **var** :

```
var nbnotes; //nbnotes vaut undefined  
var moyenne; //idem pour moyenne
```

ou encore :

```
var nbnotes, moyenne;
```

Il est possible de combiner la déclaration et l'affectation :

```
var msg = "Salut";  
var i = 0, j = 0, k = 0;
```

En JavaScript il est possible de re-déclarer une même variable.

Pour déclarer une variable, on peut omettre l'instruction **var**. Dans ce cas, si la variable n'a pas déjà été déclarée, elle est créée mais sa portée sera globale.

Portée

Contrairement à des langages comme le C, C++ et Java, JavaScript ne définit pas de base de portée au niveau des blocs. Toutes variables déclarées dans une fonction sont définies dans l'ensemble de la fonction.

```
function foo() {  
  for (var i = 0; i<5; i++)  
    console.log("Dans la boucle");  
  console.log(i); //Affichera 5  
};  
console.log(i); //undefined
```

Une variable a soit une portée :

- globale : définie n'importe où dans le code JavaScript
- locale : définie à l'intérieur du corps d'une fonction.

```
var v1 = "globale";  
function foo() {  
  var v2 = "locale";  
  console.log(v1); // Affichera "globale"  
  console.log(v2); // Affichera "locale"  
};
```

Types de données

Les bases de JavaScript

Types de données

Les programmes en JavaScript manipulent des valeurs, et ces valeurs appartiennent toutes à un type.

Notions d'identité et d'égalité

JavaScript distingue l'identité (égalité forte) et l'égalité

Pour tester l'égalité, on utilise l'opérateur de comparaison **==**. Celui-ci teste si deux données ont la même valeur à la conversion prêt.

Pour tester l'identité, on utilise l'opérateur de comparaison **===** qui permet de tester si deux données ont la même valeur sans conversion.

```
var n1 = 10;  
var n2 = "10";  
  
var c1 = (n1 == n2); // Test d'égalité  
// c1 contient true  
  
var c2 = (n1 === n2); // Test d'identité  
// c2 contient false
```

JavaScript définit également la non identité **!==** ainsi que la non égalité **!=**.

Types primaires et types de références

En JavaScript on distingue deux types de données

Type primaire

Un type primaire a une taille fixe en mémoire qui correspond à la plus grande taille nécessaire pour représenter un type primitif : en l'occurrence, les nombres (8 octets). Si chaque variable réserve 8 octets, il est alors possible d'y stocker indifféremment l'un des types primaires.

Type référence

Les objets (objets, tableaux, fonctions, etc.) eux ne sont pas de taille fixe. Leur valeur ne peut être stockée dans une variable de 8 octets. Il est nécessaire de passer par une référence qui va représenter l'endroit où est stockée l'information en mémoire. C'est une forme de pointeur.

Types primaires et types de références

Impact sur la manipulation de données

```
var a = 3.14; // ici a est un type primitif
var b = a;
a = 4;
console.log(b); // Affiche 3.14
```

Ici b manipule une copie de a car ce sont des variables de type primaire.

```
var a = { data: 3.14 }; // ici a est un objet (type
reference)
var b = a;
a.data = 4;
console.log(b.data); // Affiche 4
```

Cette fois b est une référence à a et toute modification de a est visible depuis b.

Il en va de même pour les arguments de fonctions.

Lorsqu'un paramètre est un objet (objet, tableau, fonction, etc.), celui-ci est passé par référence. Si celui-ci est de type primaire, il est passé par copie.

En JavaScript, les chaînes de caractères sont immuables. A ce titre, elles se comportent comme des types primaires mais de taille variable.

Les types primitifs

Nombres

Chaînes de caractères

Booléens

Les nombres

Généralités

Les nombres en JavaScript sont « des valeurs au format IEEE 754 en double précision 64 bits », d'après la spécification. Les conséquences de ce choix sont intéressantes. Il n'y a par exemple pas de type entier en JavaScript, donc il vous faudra faire preuve d'un petit peu de prudence avec les opérations arithmétiques si vous avez l'habitude de faire des maths en C ou en Java. Dans la pratique, les valeurs entières sont traitées comme des entiers représentés sur 32 bits (certaines implémentations stockent également les entiers sur 32 bits). Cela est important quand on souhaite effectuer des opérations en binaire.

À lire : http://www.w3schools.com/js/js_numbers.asp

Opérateurs numériques standard

Tous les opérateurs numériques standards sont gérés tels que l'addition, la soustraction, le modulo, etc.

```
Math.sin(3.5);  
var aire=Math.PI*r*r;
```

Conversion

On peut convertir une chaîne de caractères en un nombre entier à l'aide de la fonction `parseInt()`.

```
parseInt("123",10); // 123
parseInt("010",10); //10
parseInt("010");    // 8 - chaîne traitée comme un
nombre octal à cause du zéro initial
```

Pour convertir un nombre binaire en un entier, il suffit de préciser la base comme second argument.

```
parseInt("11",2); // 3
```

Attention :

`parseFloat()` et `parseInt()` traitent une chaîne de caractères jusqu'à ce qu'elles atteignent un caractère qui n'est pas valide pour le format numérique indiqué, puis renvoient le nombre traité jusqu'à ce point. L'opérateur `+` convertit simplement la chaîne à `NaN` à partir du moment où la chaîne contient au moins un caractère non valide.

Les nombres - Valeurs spéciales

NaN

NaN(Not a Number) est renvoyée si la chaîne est non numérique :

```
parseInt("coucou",10); // NaN
```

NaN est *toxique* : si cette valeur est fournie en entrée pour n'importe quelle opération mathématique, le résultat sera également **NaN** :

```
NaN + 5; // NaN
```

Cette valeur peut être détectée grâce à la fonction **isNaN()**

Infinity et -Infinity

JavaScript dispose également de valeur spéciales pour l'infini (**Infinity**) et l'infini négatif (**-Infinity**) :

```
1/0; // Infinity  
-1/0; // -Infinity
```

Il est possible de tester les valeurs **Infinity**, **-Infinity** et **NaN** à l'aide de la fonction native

isFinite() :

```
isFinite(1/0);      // false  
isFinite(-Infinity); // false  
isFinite(NaN);      // false
```

Les chaînes de caractères

Création d'une chaîne de caractères

Séquences de caractères Unicode, chaque caractère étant représenté par un nombre de 16 bits.

On peut créer des chaînes de caractères en utilisant des quotes simples ou des doubles quotes.

```
'toto'  
"truc"
```

Il est possible de créer des chaînes de caractères avec une notation littérale ou avec l'objet String.

```
var s1 = "toto"; // crée une valeur  
primitive qui est une chaîne  
var s2 = new String("toto"); // crée un objet String  
console.log(s1); // Affiche "toto" dans la  
console  
typeof s1; // Renvoie "string"  
console.log(s2); // Affiche String ["t",  
"o", "t", "o"]  
typeof s2; // Renvoie 'object'
```

Attention, l'objet **String** ne se comporte pas comme le type primitif chaîne.

```
var s1 = "2 + 2"; // crée une chaîne  
primitive  
var s2 = new String("2 + 2"); // crée un objet String
```

```
eval(s1);           // renvoie le nombre 4  
eval(s2);           // renvoie la chaîne "2"  
+ 2"
```

Les chaînes de caractères

Opérations sur les chaînes de caractères

Concaténer deux chaînes de caractères

Pour concaténer deux chaînes de caractères, on utilise l'opérateur **+**

```
var chaine1 = "Salut";  
var chaine2 = "John";  
var chaine3 = chaine1 + " " + chaine2    // vaut :  
Salut John
```

Connaître la longueur d'une chaîne : propriété `length` de l'objet **String**

```
var maChaîne = "Hello world !";  
var x = maChaîne.length;    // 13
```

Accéder au $i^{\text{ème}}$ caractère d'une chaîne

```
'chat'.charAt(2);    // renvoie "a"  
'chat'[2];          // renvoie "a", disponible depuis  
ECMAScript 5
```

Quelques fonctions pratiques sur les chaînes :

charAt, **charCodeAt**, **codePointAt**, **indexOf**,
lastIndexOf, **startsWith**, **endsWith**, **includes**,


```
concat, fromCharCode, fromCodePoint, split,  
slice, substring, substr, match, replace,  
search, toLowerCase, toUpperCase, normalize,  
repeat, trim
```

Booléen

En utilisant le type primitif

En JavaScript, il existe le type primitif **boolean**

```
var is_true = false;
```

Il existe aussi une fonction **Boolean()** qui permet de convertir un autre type en **boolean**

```
var functionTrue = Boolean(true);  
var functionFalse = Boolean(false);
```

Booléen

En utilisant un objet

Il est aussi possible d'utiliser la fonction **Boolean** comme un constructeur

```
var constructorTrue = new Boolean(true);  
var constructorFalse = new Boolean(false);
```

Attention dans ce cas à ne pas confondre la valeur d'un objet **Boolean** et les valeurs primitives **true** et **false**

```
var x = new Boolean(false);  
if (x) { // Boolean(x) vaut true car c'est un objet  
    bien défini, il faut utiliser x.valueOf() ici  
        // . . . le code est exécuté  
}
```

```
var x = false;  
if (x) { // Boolean(x) vaut false, c'est le type  
    primitif  
        // . . . le code n'est pas  
    exécuté  
}
```

Tout comme pour le type primitif **boolean**, il existe des équivalents objets pour les nombres (**Number**) et les chaînes de caractères (**String**). Cela rend par exemple

possible d'accéder à une propriété **length** d'une chaîne définie par un type primitif.

Les types références

Objets

Tableaux

Fonctions

Objets

En JavaScript un objet est une collection non ordonnée de propriétés. Les valeurs nommées contenues dans un objet peuvent être des valeurs primitives ou peuvent être elles mêmes des objets.

Création d'un objet

Un objet peut être créé en utilisant un littéral

```
// Littéral objet (aussi appelé initialiseur d'objet)
{ [ nomValeurPaire[, nomValeurPaire2[,
...nomValeurPaireN] ] ] }

// Appelé comme un constructeur
var livre = {}; // objet vide
```

ou encore avec l'instruction **new**

```
// Appelé comme un constructeur
new Object([valeur])
```

Objets

Propriété d'un objet

Pour ajouter des propriétés à un objet, on peut le faire via le littéral

```
function move(point, vec) {  
    point.x +=vec.x;  
    point.y +=vec.y;  
}  
  
var point = {x:2, y:3, color: 'blue'};  
move(point, {x:0, y:1});  
console.log(point);    // Object {x: 2, y: 4, color:  
"blue"}
```

Ou alors, via l'opérateur `.`

```
var personne = new Object();  
personne.prenom = "John";  
personne.nom = "Deuf";  
personne.age = 12;  
personne.ne_le = {j:19, m:05, a:1995};
```

Objets

Propriété d'un objet

Pour vérifier l'existence d'une propriété, on peut utiliser l'instruction **in**.

```
if ("color" in point) point.color = "red";
```

Attention, même si la propriété existe, elle peut être indéfinie et aura comme valeur **undefined**.

L'instruction **delete** permet de supprimer une propriété d'un objet

```
delete point.color;
```


Tableaux

- Les tableaux sont des objets natifs
- Création d'un tableau
En JavaScript il y a deux solutions pour déclarer un tableau :
 - le constructeur **Array**
 - et une expression littérale.

Tableaux

Expression littérale

Liste d'éléments entre crochets, séparée par des virgules.

```
// Tableau vide
var vide = [];

// Tableau avec 7 éléments de type chaîne de
caractères
var jours = ["lundi", "mardi", "mercredi", "jeudi",
"vendredi", "samedi", "dimanche"];

// 3 éléments de type différents
var divers = [1.1, true, "a"];
```

Les tableaux littéraux peuvent contenir des objets ou encore des expressions quelconques

```
var base = 1024;
var table = [base, 2*base, {x:1, y:2}];
```

Tableaux

Constructeur Array

Utilisation de l'instruction **new**.

```
// Tableau vide
var vide = new Array();

// Tableau avec 7 éléments de type chaîne de
caractères
var jours = new Array("lundi", "mardi", "mercredi",
"jeudi", "vendredi", "samedi", "dimanche");

// 3 éléments de type différents
var divers = new Array(1.1, true, "a");
```

On peut aussi appeler le constructeur avec un seul argument numérique

```
var tab = new Array(10); // Créé un tableau de 10
cases
```

Tableaux

Lire et écrire des éléments dans un tableau

Pour accéder aux éléments d'un tableau, on utilise l'opérateur **[]**

```

valeur = a[0];
a[1] = 3.14;
i = 2;
a[i] = 3;
a[i+1] = "Salut";
a[a[i]] = a[0];

```

En JavaScript, le premier élément est à l'indice 0.

Ajouter de nouveaux éléments

En JavaScript, un tableau n'a pas un nombre fixe d'éléments.

Pour ajouter un nouvel élément, il suffit de lui affecter une valeur.

```

a[10] = 5;

```

Tableaux

Taille d'un tableau

Un tableau peut être clairsemés. La taille d'un tableau, accessible depuis la propriété **length**, correspond à la valeur immédiatement supérieure à son plus grand indice.

Exemples

```
var a = new Array();           // a.length == 0
a = new Array(10);             // a.length == 10
a = new Array(1, 2, 3);        // a.length == 3
a = [4,5];                     // a.length == 2
a[5] = -1;                     // a.length == 6
a[49] = 0;                     // a.length == 50
```

Fonctions

Les fonctions sont des objets “classiques” que l’on peut “appeler”.

Définir une fonction

Il y a trois façons de définir une fonction :

- l’instruction **function** ;
- le constructeur de l’objet **Function** ;
- Et une expression littéral.

Fonctions

Instruction function

```
function [nom]([param] [,param] [...,param]) {  
    instructions  
}
```

nom : Le nom de la fonction.

param : Le nom d'un argument à passer à la fonction (une fonction pouvant avoir jusqu'à 255 arguments).

instructions : Les instructions qui forment le corps de la fonction.

Fonctions

L'objet Function

```
new Function ([arg1[, arg2[, ...argN]],  
corpsFonction)
```

arg1, arg2, ... argN

Nom des paramètres formels sous la forme d'une chaîne de caractères correspondant à un identifiant JavaScript valide. Par exemple : "x", "uneValeur", ou "a,b".

corpsFonction

Chaîne de caractères contenant les instructions JavaScript définissant la fonction.

Exemple

```
var ajoute = new Function('a', 'b', 'return a + b');  
ajoute(2, 6); // Retourne 8
```


Fonctions

Expression littérale

La fonction est utilisée comme une expression et produit en fait un Objet **function**

```
nom = function ([param[, param[, ... param]]) {  
    instructions  
}
```

nom : Le nom de la fonction. Il est facultatif, auquel cas la fonction devient une fonction anonyme.

param : Le nom d'un argument à passer à la fonction (une fonction pouvant recevoir jusqu'à 255 arguments).

instructions : Les instructions qui forment le corps de la fonction.

Fonctions

L'instruction `return`

L'instruction `return` met fin à l'exécution d'une fonction et définit une valeur à renvoyer à la fonction appelante.

RETOURNER UNE VALEUR

```
function carre(x) {  
    return x * x;  
}
```

METTRE FIN À UNE FUNCTION

```
function compteur() {  
    for (var compte = 1; ; compte++) { // boucle  
        infinie  
            if (compte === 5) {  
                return;  
            }  
        }  
        console.log(compte); // cette instruction n'est  
        jamais utilisée  
    }  
    compteur();  
}
```

RETOURNER UNE FONCTION

```
function bar(x) {  
    return function(x) { return x * x }; // C'est un  
    littéral  
}
```

```
var foo = bar();  
foo(2); // 4
```

Fonctions

Fonction et portée des variables

On ne peut pas accéder aux variables définies dans une fonction en dehors de cette fonction.

Soit le code ci-dessous, **id** est accessible de l'extérieur et est donc modifiable de l'extérieur de la fonction.

```
function idunique(){  
    var id=0;  
    return id++;  
};  
console.log(idunique()); // Affiche 0  
console.log(idunique()); // Affiche 0
```

Pour que cela fonctionne, on peut utiliser une variable globale (mais il y a mieux !)

```
var id=0;  
function idunique(){  
    return id++;  
};  
console.log(idunique()); // Affiche 0  
console.log(idunique()); // Affiche 1  
  
id = 0;  
console.log(idunique()); // Affiche 0
```

Fonctions

Portée et fonction interne

Il est possible en JavaScript de déclarer une fonction à l'intérieur d'autres fonctions. Dans ce cas, les fonctions peuvent accéder à des variables de leur fonction parente. On peut faire de **id** une propriété de la fonction mais il reste tout de même modifiable.

```
var idunique = function gen(){
    if (!gen.id) gen.id = 0;
    return gen.id++;
};

console.log(idunique()); // Affiche 0
console.log(idunique()); // Affiche 1

idunique.id = 0;
console.log(idunique()); // Affiche 0
```

Fonctions

Portée et fonction interne

Il est possible en JavaScript de déclarer une fonction à l'intérieur d'autres fonctions. Dans ce cas, les fonctions peuvent accéder à des variables de leur fonction parente. On peut faire de **id** une propriété de la fonction mais il reste tout de même modifiable.

La solution consiste à *cacher* **id** dans le contexte d'une fonction interne.

```
var idunique = (function(){  
    var id = 0;  
    return function() { return id++; };  
})();  
console.log(idunique()); // Affiche 0  
console.log(idunique()); // Affiche 1  
  
idunique.id = 0;  
console.log(idunique()); // Affiche 2, il ne s'agit  
pas de idunique.id
```

Fonctions

Fermeture

Wikipédia : Une fermeture ou clôture (en anglais : closure) est une fonction qui capture des références à des variables libres dans l'environnement lexical. Une fermeture est donc créée, entre autres, lorsqu'une fonction est définie dans le corps d'une autre fonction et fait référence à des arguments ou des variables locales à la fonction dans laquelle elle est définie.

Exemple de fermeture

```
function pas(nombre) {  
  function suivant(valeur) {  
    return valeur + nombre;  
  }  
  return suivant;  
}  
var pas_de_1 = pas(1);  
pas_de_1(10); // retourne 11
```

Et, avec une fonction anonyme

```
function creerAdditionneur(a) {  
  return function(b) {  
    return a + b;  
  }  
}  
var x = creerAdditionneur(5);  
var y = creerAdditionneur(20);
```

```
x(6); // renvoie 11  
y(7); // renvoie 27
```

Les fermetures permettent de créer des contextes autonomes. On s'en sert par exemple pour rendre privées des fonctions, ou encore pour créer des espaces de noms.

Fonctions

Paramètres par défaut

ES 5 permet de gérer les paramètres par défaut d'une fonction via la propriété globale **undefined**.

```
function f (x, y, z) {  
  if (y === undefined)  
    y = 7;  
  
  if (z === undefined)  
    z = 42;  
  
  return x + y + z;  
};  
  
f(1) === 50;  //true
```

Fonction a nombre de paramètres variable

L'objet **arguments** permet d'accéder aux arguments d'une fonction.

```
function f (x, y) {  
    var moreParams =  
    Array.prototype.slice.call(arguments, 2);  
    return (x + y) * moreParams.length;  
};
```

Exemple

```
function myConcat(separateur) {  
    var args = Array.prototype.slice.call(arguments, 1);  
    return args.join(separateur);  
}  
  
// renvoie "rouge, orange, bleu"  
myConcat(", ", "rouge", "orange", "bleu");  
  
// renvoie "éléphant ; giraffe ; lion ; guépard"  
myConcat(" ; ", "elephant", "giraffe", "lion",  
"guépard");
```

Instructions de contrôle du flux d'exécution

Les bases de JavaScript

Itérations

do...while

Cette instruction crée une boucle qui s'exécute tant que la condition est vraie. La condition est évaluée après avoir exécuté une itération de boucle, ce qui fait que cette boucle sera exécutée au moins une fois.

for

Cette instruction crée une boucle qui se base sur trois expressions facultatives. Ces expressions sont entre parenthèses, séparées par des points virgules et suivies par l'instruction à exécuter dans la boucle.

for...in

Cette instruction effectue, dans un ordre arbitraire, une boucle sur les propriétés énumérables d'un objet. Pour chacune des différentes propriétés, des instructions peuvent être exécutées.

while

Cette instruction permet de créer une boucle qui s'exécute tant qu'une condition de test est vérifiée. La condition est évaluée avant d'exécuter l'instruction contenue dans la boucle.

Contrôle du déroulement

break

Cette instruction termine la boucle ou l'instruction switch ou l'instruction label en cours et continue l'exécution sur l'instruction suivant l'instruction terminée.

continue

Cette instruction termine l'exécution des instructions dans la boucle courante, ou la boucle avec une étiquette correspondante, et continue l'exécution de la boucle dans l'itération suivante.

if...else

Cette instruction exécute une instruction si une condition donnée est vérifiée. Si la condition n'est pas vérifiée une autre instruction pourra être exécutée.

switch

Cette instruction permet d'évaluer une expression et de faire correspondre le résultat de cette expression avec différents cas et d'exécuter les instructions associées aux cas qui ont chacun un identifiant.

throw

Cette instruction lève une exception.

try...catch

Cette instruction permet de spécifier un ensemble

d'instructions à tenter, et de préciser le traitement à effectuer dans le cas où une exception est produite.

ES5 et la programmation orientée objets

Les bases de JavaScript

POO et JavaScript

JavaScript ne prend pas en charge la notion de classe telle que présente en C++ ou Java. Cependant, il est possible de définir des pseudo-classes au travers de constructeurs et de la notion de prototype.

Constructeurs

Pour définir un objet, on peut procéder ainsi

```
function Rectangle(l,h) {  
    this.largeur = l;  
    this.hauteur = h;  
}  
  
var r1 = new Rectangle(2,4);  
var r2 = new Rectangle(8, 11);
```


POO et JavaScript

Ajouter des méthodes aux objets

On peut ajouter des méthodes aux objets tout comme on ajouterait une propriété

```
function Rectangle(l,h) {  
    this.largeur = l;  
    this.hauteur = h;  
  
    this.aire = function () {  
        return this.largeur * this.hauteur;  
    }  
}  
  
var r = new Rectangle(2,4);  
var a = r.aire();
```

```
function Rectangle(l,h) {  
    this.largeur = l;  
    this.hauteur = h;  
}  
  
Rectangle.prototype.aire = function () {  
    return this.largeur * this.hauteur;  
}  
  
var r = new Rectangle(2,4);  
var a = r.aire();
```

Cependant, dans ce cas, chaque rectangle aura trois propriétés. Si **largeur** et **hauteur** peuvent être différentes, la façon de calculer l'air est identique pour tout rectangle.

Pour résoudre ce problème, on peut utiliser la propriété **prototype**. Cette propriété est partagée par l'ensemble des instances d'un même objet.

POO et JavaScript

Héritage

On définit la propriété prototype de notre classe fille en appelant le constructeur de la classe mère.

```
// Le constructeur Forme
function Forme(x,y) {
    this.x = 0;
    this.y = 0;
}

// Méthode de la classe parente
Forme.prototype.deplacer = function(x, y) {
    this.x += x;
    this.y += y;
    console.info('Forme déplacée.');
```

```
// Le constructeur Rectangle - classe fille
function Rectangle(x,y,l,h) {
    Forme.call(this,x,y); // on appelle le constructeur parent
    this.largeur = l;
    this.hauteur = h;
}
```

POO et JavaScript

Héritage

On redéfinit le constructeur de la propriété **prototype** comme celui de la classe fille.

```
Rectangle.prototype = Object.create(Forme.prototype);  
// On remplace le prototype de Objet  
Rectangle.prototype.constructor = Rectangle; // On  
définit le bon constructeur  
  
// Ajout d'une méthode à la classe fille  
Rectangle.prototype.air = function(){  
    return this.largeur*this.hauteur;  
}  
  
var rect = new Rectangle(0,0,2.5,5);  
var a = rect.air();  
  
console.log('rect est-il une instance de Rectangle ?  
' , (rect instanceof Rectangle)); // true  
console.log('rect est-il une instance de Forme ? ',  
(rect instanceof Forme)); // true  
  
rect.deplacer(1, 1); // Affiche 'Forme déplacée.'
```

POO et JavaScript

Polymorphisme

Il est possible de mettre en place le polymorphisme grâce à tout ce qui précède

Définition de la classe Forme

```
function Forme(x,y) {  
    this.x = 0;  
    this.y = 0;  
}
```

Définition de la classe Ellipse

```
function Ellipse(x,y,a,b) {  
    Forme.call(this,x,y); // on appelle le constructeur  
    parent  
    this.a = a;  
    this.b = b;  
}  
  
Ellipse.prototype = Object.create(Forme.prototype);  
Ellipse.prototype.constructor = Ellipse;  
  
Ellipse.prototype.perimetre = function(){  
    console.log("périmètre de Ellipse");  
    return Math.PI*Math.sqrt(2*  
(this.a*this.a+this.b*this.b));  
}
```

Définition de la classe Cercle

```
function Cercle(x,y,r) {  
    Ellipse.call(this,x,y,r,r);  
}  
  
Cercle.prototype = Object.create(Ellipse.prototype);  
Cercle.prototype.constructor = Cercle;  
Cercle.prototype.perimetre = function(){  
    console.log("périmètre du cercle");  
    return (2*Math.PI*this.a);  
}
```

POO et JavaScript

Polymorphisme

Mise en œuvre

```
var forms = new Array(2);  
forms[0] = new Cercle(0,0,1);  
forms[1] = new Ellipse(0,0,1,2);  
  
for (var i=0; i<forms.length; i++)  
    console.log(forms[i].perimetre());
```

POO et JavaScript

Méthodes privées

On peut aller encore plus loin en encapsulant nos objets dans un espace de nom. Combiné avec un littéral objet, cela permet de définir des membres privées et des membres publics (que l'on expose).

```
var Shift = (function () {  
    function isNumber(n) {  
        return (typeof n) === 'number';  
    }  
  
    function shiftIt(n) {  
        if (isNumber(n))  
            return n+1;  
        return false;  
    }  
  
    return {  
        next: shiftIt  
    };  
})();  
  
console.log(Shift.next(3));    //4  
console.log(Shift.next("1")); //false
```


POO et JavaScript

Méthodes privées

Il est aussi possible de passer des paramètres à la construction.

```
var Shift = (function (d){
    var delta = d || 1;
    function isNumber(n) {
        return (typeof n) === 'number';
    }

    function shiftIt(n) {
        if (isNumber(n))
            return n+delta;
        return false;
    }

    return {
        next: shiftIt
    };
})(Math.floor(10*Math.random()+1));

console.log(Shift.next(3));    //dépend du nombre
                               //aléatoire
console.log(Shift.next("1")); //false
```

Conclusion

Les bases de JavaScript

Le mot de la fin

JavaScript ne se limite pas à un usage dans une page web, c'est avant tout un langage de script (cf. **nodejs**).

Quelques liens à aller voir :

- [**https://nodejs.org/en/**](https://nodejs.org/en/)
- [**https://angularjs.org/**](https://angularjs.org/)
- [**http://davidbcalhoun.com/2014/what-is-amd-commonjs-and-umd/**](http://davidbcalhoun.com/2014/what-is-amd-commonjs-and-umd/)
- [**https://addyosmani.com/writing-modular-js/**](https://addyosmani.com/writing-modular-js/)
- [**https://www.linkedin.com/pulse/amd-vs-common-js-umd-damodaran-sathyakumar**](https://www.linkedin.com/pulse/amd-vs-common-js-umd-damodaran-sathyakumar)

Sans oublier : [**https://developer.mozilla.org/en-US/docs/Web/JavaScript**](https://developer.mozilla.org/en-US/docs/Web/JavaScript)

Speaker notes

dernière mise à jour le 02/09/2024
LES BASES DE JAVASCRIPT / B. Desbenoit